TrailVerse Relaunch — Product Requirements Document

Version: 1.0 **Author:** Krishna Sathvik Mantripragada **Date:** March 28, 2026 **Status:** Draft

---

Executive Summary

TrailVerse is repositioning from an "AI-powered trip planner" to **the best modern platform for exploring America's National Parks**. This relaunch involves three workstreams: removing guest access restrictions to enable full open browsing, migrating from Vite + React SPA to Next.js for SEO-critical server-side rendering, and refreshing the landing page copy to reflect the exploration-first positioning.

Core thesis: Let the content be the product. 470+ parks, real-time weather, events, interactive maps, and community reviews should be fully accessible to everyone. AI trip planning and personal features become the conversion lever, not the gate.

Workstream 1: Remove Guest Restrictions

Problem

The current `isPublicAccess` system restricts unauthenticated users to page 1 of explore (no search, no filters, no pagination), limited map functionality, restricted compare and events pages, and a blocked Daily Feed. This creates friction at the discovery stage — the exact moment when users should be falling in love with the platform. Every competitor (NPS.gov, AllTrails, The Dyrt) lets users browse freely. TrailVerse gates content that has no reason to be gated.

Goal

Open all content and discovery features to unauthenticated users. Gate only features that require user identity (writing data, personalization, AI usage beyond free trial).

Access Model

Fully open (no auth required):

- Explore page — all 470+ parks, full search, all filters, all pagination
- Park detail pages — full info, weather, alerts, photos, activities, reading reviews
- Interactive map — all parks, markers, search, zoom, nearby places, directions
- Compare page — full side-by-side comparison of up to 4 parks
- Events page — all park events, ranger programs, filtering

- Activity detail pages — full activity information
- Blog — all posts, reading comments
- Landing page, Features page, About page, FAQ page
- Daily Feed — full read access

Gated (auth required):

- AI Trip Planner — 3 free messages for anonymous users (keep existing `anonymousSession` logic with 48-hour window), unlimited for logged-in users
- Writing reviews — creating, editing, deleting reviews and uploading review images
- Favorites — saving parks, blogs, events to favorites
- Visited parks tracking — marking parks as visited with dates and memories
- Trip history — saving, archiving, restoring trip conversations
- Profile management — avatar, bio, email preferences, password changes
- Writing blog comments and replies
- Saving events to profile
- Writing testimonials
- WebSocket real-time sync (requires authenticated session)

Implementation Plan

Pages to modify (remove `isPublicAccess` restrictions):

File	Changes
`ExploreParksPage.jsx`	Remove page 1 lock, show search bar, show filters sidebar, enable pagination for all users
`ParkDetailPage.jsx`	Remove review/favorite auth checks for viewing (keep for writing). Show "Login to write a review" and "Login to favorite" prompts on action buttons
`MapPage.jsx`	Remove page 1 restriction, enable full map features for all users
`MobileMapPage.jsx`	Same as MapPage
`ComparePage.jsx`	Remove `isPublicAccess` restrictions entirely
`EventsPage.jsx`	Remove browsing restrictions. Keep "Login to save event" on save action
`ActivityDetailPage.jsx`	Remove `isPublicAccess` restrictions entirely
`DailyFeedPage.jsx`	Remove `enabled: !!user?._id` gate. Show feed to all users. Gate interactions (likes, saves) behind auth

Component-level changes:

- Remove all blue `isPublicAccess` banners ("You're viewing the first page of X pages. Login to...")
- Replace with contextual login prompts only on gated actions (e.g., clicking the favorite heart shows a tooltip/modal: "Create a free account to save favorites")
- Keep the AI planner banner as-is ("You can chat 3 messages with our AI trip planner. Login for unlimited...")

Backend changes:

- Audit all API routes that currently require auth middleware for read operations
- Ensure `/api/parks`, `/api/parks/:parkCode`, `/api/events`, `/api/blogs`, `/api/reviews/park/:parkCode` (GET), `/api/enhanced-parks` are accessible without JWT
- Keep auth middleware on all POST/PUT/DELETE operations and on `/api/ai/chat` (with anonymous session fallback)

Auth prompt pattern:

When an unauthenticated user tries a gated action, show an inline prompt (not a redirect, not a modal wall):

```
[Heart icon] Login to save this park to your favorites
[Star icon] Login to write a review
[Bookmark icon] Login to save this event
```

Clicking these navigates to `/login?redirect={currentPage}` so users return to where they were after auth.

Success Metrics

- Bounce rate decrease on explore, park detail, and map pages
- Pages per session increase for unauthenticated users
- Signup conversion rate (users who browse → create account)
- Time on site for unauthenticated users

Workstream 2: Next.js Migration

Problem

TrailVerse is a Vite + React SPA. Every page returns the same HTML shell with "You need to enable JavaScript to run TrailVerse." Google's crawler sees no content. For a platform with 470+ unique park pages, blog posts, and activity pages, this means zero organic search traffic. Searches like "Yellowstone trip itinerary" or "Zion National Park weather" will never surface TrailVerse.

Goal

Migrate the frontend from Vite + React to Next.js with a hybrid SSR/SSG strategy. Every park, blog post, and key content page becomes a crawlable, indexable HTML page. The backend (Express + MongoDB on Render) stays untouched.

Architecture

Current:

```
[Vercel] → Vite SPA (client-rendered)
[Render] → Express API + MongoDB Atlas
```

After migration:

```
[Vercel] → Next.js (SSR/SSG + client hydration)
[Render] → Express API + MongoDB Atlas (unchanged)
```

Next.js runs on Vercel. It fetches data from the Express API on Render at build time (SSG) or request time (SSR). The Express backend, MongoDB, all API routes, middleware, services — none of that changes.

Rendering Strategy

Page	Strategy	Rationale
Landing page	SSG	Static content, changes rarely
Explore page	SSG + ISR (revalidate: 3600)	Park list is stable, refresh hourly
Park detail pages (470+)	SSG with `generateStaticParams` + ISR (revalidate: 1800)	Critical for SEO. Each park = indexable page. Weather/alerts refresh via client-side fetch
Blog listing	SSG + ISR (revalidate: 3600)	New posts are infrequent
Blog post pages	SSG with `generateStaticParams` + ISR (revalidate: 3600)	Each post = indexable page
Activity detail pages	SSG + ISR (revalidate: 86400)	Rarely changes
Map page	CSR (client-only)	Google Maps requires browser. SSR adds no SEO value for a map
Compare page	CSR	Dynamic user-driven selections, no SEO value
AI Trip Planner	CSR	Interactive, auth-gated, no SEO value
Events page	SSR (per request)	Events change frequently, need fresh data
Profile, Admin, Auth pages	CSR	Behind auth, no SEO value
Features, About, FAQ, Privacy, Terms	SSG	Static marketing pages

Migration Approach

Phase 1 — Scaffold and shared infrastructure (Days 1–5)

- Initialize Next.js 14+ project with App Router (`/app` directory)
- Port Tailwind CSS 4 config, global styles, CSS variables, theme system
- Set up environment variables for API URL, Google Maps key, GA tracking ID
- Port `AuthContext`, `ThemeContext`, `ToastContext` as client-side providers
- Create root layout with Header/Footer (mark as client components where needed)
- Set up TanStack Query provider for client-side data fetching
- Configure `next.config.js` — image domains, redirects, rewrites
- Set up path aliases to match current `@components/icons` pattern

Phase 2 — Static and marketing pages (Days 6–8)

- Migrate: Landing, Features, About, FAQ, Privacy, Terms, NotFound
- These are mostly presentational — straightforward port with `metadata` exports for SEO
- Landing page gets copy refresh (see Workstream 3)

Phase 3 — Core content pages with SSG (Days 9–16)

- Migrate Explore page with SSG + ISR
- Migrate Park detail pages — `generateStaticParams` fetches all park codes from API, each page pre-renders with park data, weather/alerts load client-side
- Migrate Activity detail pages with SSG
- Migrate Blog listing and Blog post pages with SSG
- Migrate Events page with SSR

Phase 4 — Interactive and auth pages as CSR (Days 17–22)

- Migrate Map page, Compare page as client-only (`'use client'` throughout)
- Migrate AI Trip Planner as client-only
- Migrate all auth pages (Login, Signup, ForgotPassword, ResetPassword, VerifyEmail)
- Migrate Profile page and all profile sub-components
- Migrate Admin dashboard and admin pages
- Migrate DailyFeedPage

Phase 5 — Services, hooks, and utilities (parallel with Phases 2–4)

- Port all 32 frontend services — most are API call wrappers using Axios, these stay as-is
- Port all 18 custom hooks
- Port utility functions
- Port WebSocket service (client-only)
- Port Google Maps loader and animated pin builder
- Ensure TanStack Query hooks work with Next.js hydration

Phase 6 — SEO and metadata (Days 23–25)

- Replace `react-helmet-async` with Next.js native `metadata` and `generateMetadata`
- Add `generateMetadata` to every page with dynamic titles, descriptions, and Open Graph images
- Generate `sitemap.xml` using Next.js `sitemap.ts` (dynamic, auto-includes all parks and blog posts)
- Generate `robots.txt` via Next.js config
- Add JSON-LD structured data for parks (Place schema), blog posts (Article schema), and organization
- Verify all 470+ park pages are crawlable and indexable

Phase 7 — Testing, performance, and deployment (Days 26–30)

- Run Lighthouse on key pages — target 90+ on Performance, 100 on SEO
- Test all SSG pages render correctly without JavaScript
- Test hydration — client-side features (maps, AI chat, favorites) work after page load
- Test auth flow — login redirect preserves destination
- Verify Google Search Console shows indexed pages
- Deploy to Vercel, configure custom domain, test production build
- Set up ISR revalidation webhooks (optional: trigger revalidation from admin when park data updates)

Key Technical Decisions

App Router vs Pages Router: Use App Router (`/app` directory). It's the current standard for Next.js, supports React Server Components, streaming, and native metadata API.

Data fetching pattern: SSG/ISR pages fetch from Express API at build/revalidation time using `fetch()` with Next.js caching. Client-side interactions (favorites, reviews, AI chat) use TanStack Query + Axios as they do today.

Component strategy: Most existing components are client components (they use hooks, state, browser APIs). Mark them with `'use client'`. Server Components are used for page layouts and data-fetching wrappers.

Google Maps: Stays client-only. The map page won't benefit from SSR. Wrap in `'use client'` component, load Google Maps SDK on client.

Socket.io: Stays client-only. Initialize in a `useEffect` as today.

Image optimization: Replace custom `OptimizedImage` component with Next.js `<Image>` for automatic WebP, lazy loading, and responsive sizes.

Risks and Mitigations

Risk	Mitigation
Migration introduces regressions in 74+ components	Migrate page-by-page, test each before moving to next. Keep Vite build as fallback until migration is verified
TanStack Query hydration mismatches	Use `HydrationBoundary` and `dehydrate` for SSG pages that also use client-side queries
Google Maps breaks in SSR context	Wrap all map components in `'use client'` with dynamic import (`next/dynamic` with `ssr: false`)
Build time for 470+ static pages	Next.js handles this well on Vercel. Expect 3–5 minute builds. Use ISR so pages regenerate on demand, not all at once
Environment variable conflicts	Next.js uses `NEXT_PUBLIC_` prefix for client-side vars. Rename all `VITE_` and `REACT_APP_` vars

Files and Structure

```
trailverse-next/
├── app/
|   ├── layout.tsx                    # Root layout (Header, Footer, Providers)
|   ├── page.tsx                      # Landing page (SSG)
|   ├── explore/
|   |   └── page.tsx                  # Explore parks (SSG + ISR)
|   ├── parks/
|   |   └── [parkCode]/
|   |       └── page.tsx              # Park detail (SSG + ISR,
generateStaticParams)
|   ├── activities/
```

```
|   |   └── [activityId]/
|   |       └── page.tsx              # Activity detail (SSG + ISR)
|   ├── map/
|   |   └── page.tsx                  # Map page (CSR)
|   ├── compare/
|   |   └── page.tsx                  # Compare page (CSR)
|   ├── plan/
|   |   └── page.tsx                  # AI Trip Planner (CSR)
|   ├── events/
|   |   └── page.tsx                  # Events (SSR)
|   ├── blog/
|   |   ├── page.tsx                  # Blog listing (SSG + ISR)
|   |   └── [slug]/
|   |       └── page.tsx              # Blog post (SSG + ISR,
generateStaticParams)
|   ├── feed/
|   |   └── page.tsx                  # Daily Feed (CSR)
|   ├── profile/
|   |   └── page.tsx                  # Profile (CSR, auth required)
|   ├── login/
|   |   └── page.tsx                  # Login (CSR)
|   ├── signup/
|   |   └── page.tsx                  # Signup (CSR)
|   ├── admin/
|   |   ├── page.tsx                  # Admin dashboard (CSR, admin auth)
|   |   └── .../                      # Admin sub-pages
|   ├── features/
|   |   └── page.tsx                  # Features (SSG)
|   ├── about/
|   |   └── page.tsx                  # About (SSG)
|   ├── faq/
|   |   └── page.tsx                  # FAQ (SSG)
|   ├── privacy/
|   |   └── page.tsx                  # Privacy (SSG)
|   ├── terms/
|   |   └── page.tsx                  # Terms (SSG)
|   ├── sitemap.ts                    # Dynamic sitemap
|   ├── robots.ts                     # Robots.txt
|   └── not-found.tsx                 # 404 page
├── components/                       # Ported from current client/src/components
|   ├── ai-chat/
|   ├── blog/
|   ├── common/
|   ├── explore/
```

```
│   ├── map/
│   ├── park-details/
│   ├── plan-ai/
│   ├── profile/
│   ├── reviews/
│   ├── testimonials/
│   └── icons/
├── services/                        # Ported API services (32 files)
├── hooks/                           # Ported custom hooks (18 files)
├── context/                         # Auth, Theme, Toast providers
├── utils/                           # Utility functions
├── lib/                             # Google Maps loader, pins, etc.
├── styles/                          # Global CSS, themes, blog prose
├── public/                          # Static assets, images, favicons
├── next.config.js
├── tailwind.config.js
├── tsconfig.json
└── package.json
```

Workstream 3: Landing Page Copy Refresh

Problem

The current landing page hero badge reads "AI-Powered Trip Planning" and the subheadline leads with "AI-powered guidance." The CTA references the old guest restriction ("View the first 12 parks without signing up"). This contradicts the new positioning of TrailVerse as the best modern platform for exploring national parks, with AI as a feature — not the identity.

Goal

Update the landing page copy to reflect exploration-first positioning. No layout changes, no structural redesign. Just words.

Changes

Hero badge:

Current	New
`AI-Powered Trip Planning`	`470+ Parks. One Platform.`

Hero headline: No change. "Discover America's Natural Wonders." already works.

Hero subheadline:

Current	New
"Explore 470+ National Parks with AI-powered guidance, real-time weather, and personalized recommendations."	"Explore 470+ National Parks with real-time weather, interactive maps, community reviews, and smart trip planning."
"Your perfect adventure starts here."	"Your next adventure starts here."

CTA section:

Current	New
"Start exploring now! View the first 12 parks without signing up, then create your free account to unlock full trip planning and favorites."	"Start exploring now — browse every park, check live weather, compare destinations, and plan your next trip."

CTA button text: No change. "Explore Now" is good.

SEO title:

Current	New
"TrailVerse - Explore America's 470+ National Parks & Sites"	"TrailVerse — Explore America's 470+ National Parks, Monuments & Historic Sites"

SEO description:

Current	New
"Discover, plan, and explore America's 470+ National Parks, Monuments, Historic Sites, and more with AI-powered trip planning, interactive maps, real-time weather, events calendar, and expert travel guides."	"Browse all 470+ National Parks with real-time weather, interactive maps, community reviews, park comparison, events, and AI trip planning. Free to explore — no account needed."

Schema.org description:

Current	New
"Your guide to exploring America's 470+ National Parks & Sites"	"The modern platform for exploring America's 470+ National Parks, Monuments, and Historic Sites"

Files to Modify

File	Changes
`LandingPage.jsx` (or Next.js equivalent after migration)	Hero badge text, subheadline text, CTA paragraph text, SEO metadata, schema description

What Stays the Same

- Hero background image and gradient overlay
- Headline: "Discover America's Natural Wonders."
- Demo video section
- Feature highlights section
- "How It Works" 4-step journey
- Testimonials section
- CTA button: "Explore Now"
- Layout, spacing, animations, responsive behavior

Timeline

Week	Workstream	Deliverables
Week 1	Guest restrictions + Landing copy	Remove `isPublicAccess` from all pages, update auth prompts, refresh landing copy. Ship independently of Next.js migration
Week 2	Next.js — Scaffold + Static pages	Initialize project, port shared infra, migrate marketing pages (Landing, Features, About, FAQ, Privacy, Terms)
Week 3	Next.js — Core content pages	Migrate Explore (SSG+ISR), Park detail pages (SSG+ISR), Blog pages, Activity pages, Events page

Week	Workstream	Deliverables
Week 4	Next.js — Interactive + Auth pages	Migrate Map, Compare, AI Planner, Profile, Admin, Auth pages. Port all services, hooks, utilities
Week 5	SEO, Testing, Launch	Metadata, sitemap, structured data. Lighthouse audits, hydration testing, auth flow testing. Deploy to Vercel, verify Google indexing

Note: Workstream 1 (guest restrictions) and Workstream 3 (landing copy) can ship on the current Vite stack immediately. They don't depend on the Next.js migration. Ship them first, then begin the migration.

Decisions

1. **Domain strategy** — `nationalparksexplorerusa.com` stays as primary domain. SEO-friendly for park searches. TrailVerse remains the brand name.
2. **TypeScript adoption** — Gradual. All new Next.js files (pages, layouts, new components) written in TypeScript. Existing components ported as-is in `.jsx` — no rewrite just for types. Convert files to `.tsx` over time as they're touched. Next.js handles mixed `.tsx`/`.jsx` natively.
3. **Blog & Admin CMS** — Deferred. Keep current TinyMCE admin setup for now. Blog and admin UI need separate fixes and will be addressed in a future phase after the relaunch.
4. **Analytics migration** — Switch from `react-ga4` to `@next/third-parties/google` during the Next.js migration. Add Vercel Analytics for performance monitoring. Dual setup: GA4 for product analytics, Vercel Analytics for performance.
5. **Testing framework** — Keep Playwright for E2E. Vitest for unit tests (consistent with current setup). Jest stays for server-side tests (backend is unchanged).
6. **Monitoring** — Vercel Analytics for now. Sentry can be evaluated post-launch if error visibility becomes an issue.

Open Questions

1. **Blog & Admin UI redesign** — Needs its own planning phase after relaunch. Current admin works but UI needs fixes.
2. **Mobile app** — Mentioned in the original Medium post as a future goal. Evaluate after Next.js migration is stable.